

بخش A2 — استانداردسازی فرمت Event Object

مقدمه

این سند مربوط به **مرحله A2 از فاز اول پروژه** است و ادامه‌ی مستقیم مرحله A1 محسوب می‌شود.

در A1 مشخص کردیم **چه Event‌هایی** در بازی وجود دارند. در A2 مشخص می‌کنیم **هر Event دقیقاً به چه شکلی** باید در کل سیستم (UI, Logic, EventBus) و در آینده (Server) نمایش داده شود.

هدف این مرحله ایجاد یک **قرارداد داده‌ای مشترک (Data Contract)** برای تمام Event‌ها است.

هدف مرحله A2

هدف A2 این است که: - تمام Event‌ها ساختار یکسان و قابل پیش‌بینی داشته باشند - UI, Logic و Server دقیقاً با یک فرمت صحبت کنند - از پراکندگی و ناسازگاری Event‌ها جلوگیری شود - پروژه برای فاز ۲ (Server) بدون Refactor آماده باشد

پس از پایان A2:

هیچ Eventی نباید خارج از این فرمت ساخته یا پردازش شود.

مشکل Event‌های بدون استاندارد

بدون Event، A2 ممکن است به شکل‌های متفاوتی ساخته شوند:

- در UI یک شکل
- در Logic شکل دیگر
- در Handlerها با if‌های زیاد

این وضعیت باعث: - باگ‌های پنهان - سختی توسعه تیمی - پیچیدگی شدید در اتصال به Server

A2 دقیقاً برای حل این مشکل تعریف شده است.

ساختار استاندارد Event Object

در این پروژه **تمام Event‌ها بدون استثنا** باید دارای این چهار فیلد باشند:

```
{  
  "type": "EVENT_TYPE",
```

```
"source": "SOURCE_ID",  
"target": "TARGET_ID",  
"payload": {}  
}
```

این ساختار برای همه Event ها (CMD, EVENT, SYS) یکسان است.

توضیح فیلدهای Event

1. type (اجباری)

تعریف: نوع Event که مشخص می‌کند چه اتفاقی رخ داده یا چه درخواستی ارسال شده است.

ویژگی‌ها: - همیشه یکی از constant های تعریف شده در `event_names.py` - مبنای Dispatch و Handler ها

مثال: - CMD_BUY_MINION - EVENT_DAMAGE_TAKEN - SYS_PHASE_CHANGED

2. source (اجباری)

تعریف: مشخص می‌کند Event از کجا تولید شده است.

کاربردها: Debug - Multiplayer - Replay - Server Sync

مقادیر رایج: "ui" - "player_1" - "game_logic" - "system"

3. target (اجباری، ولی قابل None)

تعریف: مشخص می‌کند Event روی چه موجودیتی اعمال می‌شود.

ویژگی‌ها: - می‌تواند ID کارت، Slot یا Player یا None باشد - حتی اگر وجود ندارد، فیلد باید حاضر باشد

مثال: - "minion_razorfen_1" - "shop_slot_2" - null

4. payload (اجباری)

تعریف: تمام داده‌های اختصاصی Event داخل payload قرار می‌گیرد.

قانون طلایی:

هیچ داده‌ای خارج از payload قرار نمی‌گیرد.

مثال‌ها:

:Damage Event

```
"payload": {  
  "amount": 1  
}
```

:Reorder Event

```
"payload": {  
  "from_index": 2,  
  "to_index": 5  
}
```

مثال کامل Event قبل و بعد از A2

قبل از A2 (غیر استاندارد)

```
{  
  "type": "DAMAGE",  
  "target_id": "razorfen_1",  
  "amount": 1  
}
```

بعد از A2 (استاندارد)

```
{  
  "type": "CMD_DEBUG_DAMAGE",  
  "source": "ui",  
  "target": "razorfen_1",  
  "payload": {  
    "amount": 1  
  }  
}
```

تفاوت Command و Logic Event در A2

Command Event (CMD_)

درخواست‌هایی هستند که UI به Logic ارسال می‌کند.

```
{
  "type": "CMD_BUY_MINION",
  "source": "player_1",
  "target": "shop_slot_2",
  "payload": {}
}
```

Logic / State Event (EVENT_)

نتیجه قطعی پردازش Logic که UI باید نمایش دهد.

```
{
  "type": "EVENT_GOLD_UPDATED",
  "source": "game_logic",
  "target": "player_1",
  "payload": {
    "current_gold": 7
  }
}
```

ساختار یکی است، معنا متفاوت است.

نقش EventBus در A2

EventBus: - هیچ منطق بازی ندارد - فقط Event Object را منتقل می‌کند - به payload یا type کاری ندارد

EventBus فقط تضمین می‌کند: - Event سالم عبور کند - ترتیب حفظ شود - محدودیت پردازش رعایت شود

خروجی نهایی مرحله A2

مرحله A2 زمانی کامل است که: - تمام Eventها دقیقاً از این فرمت استفاده کنند - هیچ Eventی با ساختار متفاوت وجود نداشته باشد - UI، Logic و Screenها فقط با `type` و `payload` کار کنند - Event Object به‌عنوان قرارداد رسمی تیم پذیرفته شود

ارتباط A2 با مراحل بعدی

- Screen (Screen Manager): A3 ها فقط Event استاندارد مصرف می کنند
- Combat: Event های Combat قابل Replay می شوند
- Server: Serialization بدون تغییر انجام می شود

جمع بندی

مرحله A2 پروژه را از یک پیاده سازی ساده به یک سیستم قابل توسعه، قابل تست و آماده شبکه تبدیل می کند.

پس از این مرحله، تیم می تواند با اطمینان وارد: Screen Manager - Recruit Phase - Combat Logic

شود.

پایان سند مرحله A2